

MAY 4, 2026 SYLLABUS

Databricks Certified Data Engineer Associate

Key Concepts — Summary & Reason Guide | All 7 Domains | 44 Concepts

7

Domains

44

Concepts

45

Exam Questions

70%

Pass Mark

Table of Contents

Domain 1 — Databricks Platform

- Lakehouse architecture
- Delta Lake and the transaction log
- OPTIMIZE and VACUUM
- Liquid Clustering
- Predictive Optimization
- Compute types
- Time travel

Domain 2 — Data Ingestion

- Auto Loader (cloudFiles)
- Key Auto Loader options
- COPY INTO
- mergeSchema vs overwriteSchema
- JDBC parallel ingestion
- Lakeflow Connect

Domain 3 — Transformation & Modeling

- Medallion architecture
- 2026 SDP API — @dp.table vs @dp.materialized_view
- Data quality expectations
- SCD Type II with create_auto_cdc_flow
- Streaming limitations
- MERGE INTO (upsert)
- Key SQL functions

Domain 4 — Lakeflow Jobs

- Trigger types — scheduled vs data-driven
- Streaming output modes
- Watermark
- Checkpoint — what happens without one
- Repair Run
- dbutils.secrets

Domain 5 — CI/CD

- Declarative Automation Bundles (DABs)
- DAB targets and variable overrides
- bundle validate before deploy
- Databricks Repos and Git workflow
- Databricks Connect

Domain 6 — Troubleshooting & Optimization

- Spark UI — Stages tab
- Data skew
- Disk spill
- Adaptive Query Execution (AQE)
- shuffle.partitions tuning
- Broadcast join

Domain 7 — Governance & Security

- Unity Catalog privilege model
- Managed vs external tables
- Column masking
- Row-level security (row filters)
- ABAC policies
- Delta Sharing
- Lakehouse Federation

Domain 1 — Databricks Platform

Delta Lake, compute types, Liquid Clustering, Predictive Optimization

Lakehouse architecture

Foundation of the entire platform

SUMMARY

A single platform that stores data in open Parquet/Delta format on cloud storage, adding ACID transactions, schema enforcement, and warehouse-grade query performance on top. Supports both BI and ML on the same data.

Why we use it

Before Lakehouse, teams maintained a separate data lake (cheap, flexible, no governance) AND a data warehouse (fast, governed, expensive). Lakehouse collapses both into one — you get open formats AND SQL reliability without maintaining two systems.

COMPARE

WITHOUT LAKEHOUSE	WITH LAKEHOUSE
Two systems to maintain. Data copied between lake and warehouse. Always stale. Double cost.	One system. One copy of data. Delta gives ACID + analytics in the same place.

must know

Delta Lake and the transaction log

How ACID works on object storage

SUMMARY

Delta Lake adds a `_delta_log` directory to every table. Every write (insert, update, delete, merge) creates a JSON entry in that log. The log is the source of truth — not the Parquet files themselves. Enables ACID, time travel, and schema enforcement.

Why we use it

Object storage (S3, ADLS, GCS) has no built-in transactions. Without the transaction log, concurrent writes would corrupt data silently. The log gives Delta ACID guarantees and time travel on top of plain cloud storage — no special database engine required.

COMPARE

PLAIN PARQUET ON S3	DELTA LAKE
No transactions. Concurrent writes corrupt files. No history. No schema enforcement.	Transaction log = ACID. Every version preserved. Time travel. Schema enforcement on every write.

must know

OPTIMIZE and VACUUM

Keeping Delta tables fast and clean

SUMMARY

OPTIMIZE compacts many small Parquet files into fewer large ones (~128MB each). ZORDER co-locates related data in files for faster filtering. VACUUM removes Parquet files older than the retention period (default 7 days) that are no longer referenced by the transaction log.

Why we use it

Frequent small writes (streaming, micro-batch) create thousands of tiny files. Reading 10,000 small files is far slower than reading 100 large ones due to file listing overhead and metadata operations. OPTIMIZE fixes this. VACUUM controls storage cost by removing old file versions that time travel no longer needs.

COMPARE

OPTIMIZE	VACUUM
Compacts files. Improves read speed. Run after bulk writes or streaming accumulation.	Removes old unreferenced files. Reduces storage cost. Default 7-day retention — cannot time-travel before VACUUM point.

must know

exam trick

Liquid Clustering

Modern replacement for ZORDER

SUMMARY

CLUSTER BY (column) in CREATE TABLE enables Liquid Clustering. Instead of static directory partitions, it incrementally clusters data by co-locating similar column values in the same files. Cluster keys can be changed without rewriting the entire table.

Why we use it

PARTITION BY creates one directory per distinct value — catastrophic for high-cardinality columns like user_id with 50M values (50M folders, each with tiny files). ZORDER requires full table rewrite every run. Liquid Clustering handles both: no directory explosion, only changed files are reclustered.

COMPARE

PARTITION BY	CLUSTER BY (LIQUID)
One folder per distinct value. Breaks at high cardinality. Hard to change partition key.	File-level clustering metadata. Handles millions of distinct values. Keys changeable anytime.

new 2026 must know

Predictive Optimization

Automated table maintenance

SUMMARY

Databricks monitors usage patterns on Unity Catalog managed tables and automatically schedules OPTIMIZE and VACUUM based on observed write frequency and file sizes. No manual cron jobs or maintenance notebooks needed.

Why we use it

Without it, engineers must schedule OPTIMIZE and VACUUM separately for every table. As the number of tables grows this overhead becomes significant. Predictive Optimization eliminates this by observing when a table actually needs compaction — not running maintenance when nothing changed.

new 2026

Compute types

Right compute for right workload

SUMMARY

All-purpose clusters are persistent and multi-language for interactive notebook development. Job clusters are created per run and auto-terminate for production ETL — most cost-efficient. SQL Warehouses are dedicated SQL/BI compute. Serverless starts in seconds with zero infrastructure configuration.

Why we use it

Using an all-purpose cluster for every workload wastes money — it stays running even when idle. Job clusters terminate on completion so you only pay for active processing time. SQL Warehouses are optimised for BI query patterns with auto-scaling that would be inefficient on general Spark clusters.

COMPARE

JOB CLUSTER	SQL WAREHOUSE
Best cost for production ETL. Created per run, auto-terminates after job.	Best for SQL/BI. No Spark overhead, purpose-built, auto-scales per query.

must know

Time travel

Querying and restoring historical data

SUMMARY

Every Delta write creates a new version in the transaction log. Query any historical version using VERSION AS OF or TIMESTAMP AS OF. RESTORE TABLE reverts the table to any past version by writing a new log entry pointing to that historical state.

Why we use it

Data engineers need to recover from bad writes, audit historical states, reproduce past analytical results, and debug data quality issues. Without time travel this requires manual backups. Delta provides it automatically from the transaction log with no additional storage or pipeline configuration.

must know

Domain 2 — Data Ingestion

Auto Loader, COPY INTO, Lakeflow Connect, JDBC, schema management

Auto Loader (cloudFiles)

Scalable incremental file ingestion

SUMMARY

Uses `.format('cloudFiles')` to read new files from cloud storage incrementally. A checkpoint tracks every file processed so each file is ingested exactly once. Supports directory listing (default) or file notification mode using SQS/Event Grid for billion-file scale.

Why we use it

Without Auto Loader, ingesting new files requires listing the entire directory each run — expensive at scale and error-prone for deduplication tracking. Auto Loader handles file tracking, schema inference persistence, and exactly-once semantics automatically without custom bookkeeping code.

COMPARE

DIRECTORY LISTING

Scans entire folder each run. Works up to ~millions of files before becoming slow.

FILE NOTIFICATION

Cloud events (SQS/Event Grid) push new file alerts. Scales to billions of files instantly.

must know

Key Auto Loader options

Production-ready configuration

SUMMARY

`schemaLocation` persists inferred schema across restarts. `inferColumnTypes=false` reads all columns as strings to prevent type drift. `rescuedDataColumn` captures unexpected fields as JSON instead of failing. `schemaEvolutionMode=addNewColumns` auto-adds new source columns to the target table.

Why we use it

Each option solves a specific production problem. Without `schemaLocation`, schema is re-inferred on every run — inconsistent types break downstream tables. Without `rescuedDataColumn`, a single mismatched field silently drops data or crashes the entire pipeline with no visibility into what was lost.

must know

exam trick

COPY INTO

Idempotent batch file loading

SUMMARY

A SQL command that loads files from cloud storage into a Delta table and records which files were loaded in the Delta transaction log. Re-running it safely skips already-loaded files — no duplicates are ever created regardless of how many times it is executed.

Why we use it

Batch ETL jobs sometimes need to be re-run after failures. COPY INTO makes re-runs safe by remembering exactly which files were loaded. It is ideal for one-time bulk loads or moderate-volume scheduled loads where SQL simplicity is preferred over building a full Structured Streaming pipeline.

COMPARE

AUTO LOADER

Streaming API. Checkpoint-based. Best for continuous or very high-volume ingestion.

COPY INTO

SQL command. Delta-log-tracked. Best for batch loads, one-time migrations, moderate volumes.

must know

mergeSchema vs overwriteSchema

Controlled schema changes on write

SUMMARY

`mergeSchema=true` adds new columns from the source DataFrame to the existing table schema while keeping all existing data intact. `overwriteSchema=true` replaces the entire schema AND all existing data with the DataFrame's schema — a destructive operation.

Why we use it

Production tables evolve — new columns are added at the source over time. `mergeSchema` lets the table adapt without losing existing rows or requiring a table rebuild. `overwriteSchema` is used only when a complete schema reset is intentional, such as during a major table redesign.

COMPARE

MERGESCHEMA=TRUE

Additive. Existing data preserved. New columns added. Safe for production.

OVERWRITESCHEMA=TRUE

Destructive. Entire schema and all data replaced by DataFrame. Use with caution.

exam trick

JDBC parallel ingestion

Reading large relational tables efficiently

SUMMARY

`spark.read.jdbc()` reads from relational databases via JDBC. For large tables, set `partitionColumn` (a numeric/date column), `lowerBound`, `upperBound`, and `numPartitions` to split the table into N parallel sub-queries each reading a range of values simultaneously.

Why we use it

A single JDBC query is sequential — it reads one row at a time through a single connection. For a 100M-row table this takes hours. With `numPartitions=8`, Spark issues 8 parallel queries each reading 12.5M rows simultaneously across 8 executor tasks — drastically faster.

must know

Lakeflow Connect

Zero-code enterprise source connectors

SUMMARY

A managed connector service for Salesforce, SAP, Workday, databases, and other enterprise sources. Handles OAuth authentication, incremental sync (only changed records since last run), schema mapping to Delta format, error recovery, and Unity Catalog governance — all automatically.

Why we use it

Building a custom Salesforce connector requires OAuth flows, API pagination, rate limiting, incremental change tracking, error handling, and schema evolution — weeks of engineering. Lakeflow Connect handles all of it out of the box, letting engineers focus on transformation logic rather than connector plumbing.

new 2026

must know

Domain 3 — Transformation & Modeling

Medallion architecture, LDP/SDP 2026 API, expectations, SCD Type II

Medallion architecture

Layered data quality model

SUMMARY

Bronze stores raw data exactly as received from the source — append-only, no transformations. Silver applies cleansing, type casting, deduplication, and validation. Gold contains aggregated KPIs and business-ready tables for BI and ML. Each layer adds quality without deleting the previous layer.

Why we use it

Without layered architecture, a bug in a transformation silently corrupts the only copy of data with no way to recover. Medallion keeps the raw Bronze layer untouched — if Silver logic changes, you reprocess from Bronze. Each layer is independently reprocessible. Clean separation of concerns at every stage.

COMPARE

BRONZE	SILVER / GOLD
Raw as-is. Append-only. Never modified. Source of truth for reprocessing.	Cleansed / aggregated. Always rebuildable from Bronze if logic needs to change.

must know

2026 SDP API — @dp.table vs @dp.materialized_view

Correct decorator determines processing mode

SUMMARY

@dp.table is for streaming tables backed by readStream — they process new records incrementally on each pipeline trigger. @dp.materialized_view is for batch-computed tables using dp.read() — fully recomputed on each run. @dp temporary_view creates a staging view not saved to the catalog.

Why we use it

Using @dp.table on a batch dp.read() causes a runtime error because Spark expects a streaming source. Using @dp.materialized_view on a streaming source loses the incremental processing benefit. Choosing the right decorator is what determines whether the table processes new records only or recomputes everything each run.

COMPARE

@DP.TABLE	@DP.MATERIALIZED_VIEW
Has readStream source. Incremental. Appends only new records each run.	Uses dp.read(). Batch. Full recompute from scratch every pipeline run.

new 2026

must know

exam trick

Data quality expectations

Three behaviours for three severity levels

SUMMARY

@dp.expect keeps the row and records the violation count as a pipeline metric — warn only. @dp.expect_or_drop removes the bad row and continues processing all other rows. @dp.expect_or_fail halts the entire pipeline immediately when any row violates the constraint.

Why we use it

Not all data quality violations are equally serious. A slightly unusual discount value is worth logging but not worth losing the entire order. A NULL order_id makes the row completely unprocessable — drop it cleanly. A NULL in a critical financial contract column should stop everything before bad data propagates to Gold and BI.

COMPARE

EXPECT	EXPECT_OR_DROP
Keep row + record violation count in metrics. For monitoring-only rules.	Remove bad row, continue pipeline. For non-critical attribute validation.

must know

exam trick

SCD Type II with create_auto_cdc_flow

Tracking dimension attribute history over time

SUMMARY

SCD Type II preserves full history of attribute changes. When a customer's segment changes, the old row gets __END_AT set to the change timestamp and a new row is inserted with __END_AT = NULL. Active (current) records always have __END_AT IS NULL.

Why we use it

Without SCD Type II, historical analysis is wrong. A customer was Consumer in 2024 and Corporate in 2025 — their 2024 orders should show Consumer segment, not Corporate. SCD Type II preserves this so you can always join dimensions to the state that was current at the time of each transaction.

must know

Streaming limitations

What breaks on a streaming DataFrame

SUMMARY

ROW_NUMBER() window function fails on streams because it requires seeing all rows globally before assigning ranks — impossible in micro-batch processing. monotonically_increasing_id() also fails. Use dropDuplicates(['key_column']) for streaming-safe deduplication instead.

Why we use it

Structured Streaming processes data as micro-batches — each batch sees only a small window of records, never the complete dataset. Any operation requiring a global view of all rows (ranking, sequential IDs across all time) fundamentally cannot work in this model.

COMPARE

ROW_NUMBER()	DROPDUPPLICATES(['KEY'])
Needs full global dataset. Not supported on streaming DataFrames.	Works within micro-batches. Streaming-safe deduplication method.

exam trick

must know

MERGE INTO (upsert)

Update existing rows and insert new ones atomically

SUMMARY

MERGE INTO compares a source DataFrame against a target Delta table on a key condition. Matched rows are updated, unmatched source rows are inserted, and optionally matched rows can be deleted. The entire operation is atomic — either all changes commit or none do.

Why we use it

INSERT INTO only appends — running it twice on the same data creates duplicates. MERGE INTO is idempotent for upsert patterns: if the row exists, update it; if it does not, insert it. Essential for slowly changing dimensions, CDC-based pipelines, and any pattern where source data may contain updates to existing records.

must know

Key SQL functions

Exam favourites — COALESCE, NULLIF, PIVOT

SUMMARY

COALESCE returns the first non-null value from a list of arguments. NULLIF returns NULL if both arguments are equal, otherwise returns the first argument. PIVOT converts distinct row values into columns. YEAR(), EXTRACT(YEAR FROM col), and DATE_PART('year', col) are all equivalent in Spark SQL.

Why we use it

These functions solve real data problems: COALESCE fills missing values from a priority list of fallback columns in one expression. NULLIF converts sentinel values (0, empty string) to NULL so downstream NULL-handling logic works correctly. PIVOT creates cross-tab reports without complex CASE statement chains.

exam trick

Domain 4 — Lakeflow Jobs

Triggers, streaming modes, watermark, checkpoint, retries

Trigger types — scheduled vs data-driven

Choosing the right pipeline activation method

SUMMARY

Scheduled triggers fire at a fixed clock time via cron expression regardless of data availability. File arrival triggers fire when a new file appears at a monitored cloud storage path. Table update triggers fire when a monitored Delta table receives a new committed data write.

Why we use it

Scheduled triggers run even when no new data exists — wasting compute on empty pipeline runs. Data-driven triggers fire only when actual data is available, making pipelines reactive rather than time-bound. Table update is particularly powerful for chaining pipelines: downstream runs automatically when upstream finishes, regardless of how long upstream takes.

COMPARE

SCHEDULED (CRON)	TABLE UPDATE TRIGGER
Simple. Runs at fixed time. May run even if no new data.	Data-driven. Fires only when upstream Delta table has new commits.

must know

Streaming output modes

Controlling what gets written each trigger

SUMMARY

Append mode writes only new rows to the sink — used for non-aggregation streaming queries. Update mode writes only changed rows — recommended for aggregations with watermark writing to Delta. Complete mode rewrites the entire aggregation result set every trigger — only practical for small aggregations writing to memory sinks.

Why we use it

Writing the full result set (Complete) every micro-batch on a large Delta table is extremely expensive and creates massive write amplification. Update mode writes only rows whose aggregated values changed — far cheaper. Append mode on an aggregation would produce duplicate rows as the same aggregation updates over time.

COMPARE

APPEND	UPDATE
New rows only. Use for pure row-level transforms — no aggregation.	Changed rows only. Use for aggregations with watermark. Most efficient for streaming to Delta.

must know

exam trick

Watermark

Bounding state memory for late-arriving data

SUMMARY

`withWatermark('timestamp_col', '10 minutes')` tells Spark to wait up to 10 minutes for late-arriving records before finalising an aggregation window and clearing that window's state from executor memory. Data arriving beyond the threshold is dropped.

Why we use it

Without watermark, Spark must keep state for every time window forever to handle arbitrarily late data — executor memory grows unbounded and the pipeline eventually crashes with OOM. Watermark caps memory usage by defining a maximum late-arrival tolerance, trading some late-data completeness for stable memory consumption.

must know

Checkpoint — what happens without one

Exactly-once fault tolerance

SUMMARY

A checkpoint stored in cloud storage (S3, ADLS, GCS) records stream progress — which files or Kafka offsets were processed and the current aggregation state. Without it, a cluster restart causes the stream to reprocess from the beginning of the source, producing duplicate records in the output.

Why we use it

Cloud clusters can be terminated at any time — spot instance eviction, maintenance windows, OOM kills. Without checkpointing, every failure means starting over — data is reprocessed and duplicated in the output table. Checkpointing makes streams fault-tolerant by persisting enough state to resume from the exact failure point.

must know

exam trick

Repair Run

Most efficient failure recovery

SUMMARY

When a multi-task Lakeflow Job fails midway, Repair Run reruns only the failed and downstream-skipped tasks while preserving the outputs of already-succeeded tasks. `max_retries=2` means 3 total attempts: 1 original plus 2 retries.

Why we use it

Re-running an entire 10-task pipeline because task 7 failed wastes the compute of tasks 1-6 that succeeded. Repair Run is surgical — it resumes from the failure point. Critical for long-running pipelines where early tasks (Bronze ingestion, large joins) are expensive and should not be repeated unnecessarily.

must know

dbutils.secrets

Secure credential management at runtime

SUMMARY

`dbutils.secrets.get(scope, key)` retrieves credentials stored in Databricks Secret Scopes at runtime. The secret value is automatically masked in all notebook outputs and driver logs — it is never displayed in plaintext regardless of print statements.

Why we use it

Hardcoding passwords directly in notebooks is a security violation — the notebook is visible to anyone with workspace access and credentials appear in version control history. Secret Scopes decouple credentials from code so the same notebook runs in dev and prod with different credentials injected securely at runtime.

must know

Domain 5 — CI/CD

Declarative Automation Bundles, DAB CLI, Repos, Databricks Connect

Declarative Automation Bundles (DABs)

Infrastructure-as-code for Databricks

SUMMARY

YAML-based framework (formerly Databricks Asset Bundles) for defining jobs, pipelines, clusters, and notebooks as version-controlled code. The root file `databricks.yml` defines the bundle. Deploy with `bundle deploy`, trigger execution with `bundle run`, validate syntax with `bundle validate`, remove resources with `bundle destroy`.

Why we use it

Without DABs, deploying a job to production means manually clicking through the Databricks UI — error-prone, not auditable, impossible to reproduce, and impossible to review in a pull request. DABs codify the entire deployment as YAML so any environment can be reproduced from Git history and every change is tracked.

new 2026 **must know**

DAB targets and variable overrides

Same codebase, different environments

SUMMARY

The targets block in `databricks.yml` defines environment-specific overrides. Dev target might use a small cluster and a dev catalog. Prod target uses a large cluster and prod catalog. Job/pipeline code (notebooks, Python scripts) is identical across environments — only infrastructure configuration variables differ.

Why we use it

Maintaining separate codebases for dev and prod causes environment drift — fixes applied in dev get forgotten in prod, prod-specific workarounds leak into dev. DAB targets enforce a single source of truth: one set of YAML files deployed everywhere, with only infrastructure variables changing per environment.

COMPARE

WITHOUT TARGETS

Separate configs per env. Drift inevitable. Manual edits cause mistakes.

WITH TARGETS

One codebase. Variables override per env. Guaranteed consistency across all environments.

new 2026

bundle validate before deploy

Catch YAML errors before touching production

SUMMARY

`databricks bundle validate` checks all YAML files in the bundle for syntax errors, invalid resource configurations, undefined variable references, and logical inconsistencies — without deploying anything to any workspace.

Why we use it

A YAML typo in a job definition deployed directly to production can break a live pipeline. Running `validate` in CI before `deploy` catches these errors in seconds without any production impact. Same principle as compiling code before shipping — find errors early when they are cheap to fix.

new 2026

Databricks Repos and Git workflow

Version control for notebooks and code

SUMMARY

Databricks Repos syncs notebooks and files with Git repositories (GitHub, GitLab, Azure DevOps, Bitbucket). Engineers create feature branches, commit changes, push to remote, and create pull requests for peer review before merging to the main branch.

Why we use it

Classic workspace notebooks have no Git integration — changes are not tracked, there is no history, and collaboration requires manually sharing notebook exports. Repos brings standard software engineering practices (branching, PR review, version history, rollback) to data engineering workflows.

new 2026

Databricks Connect

Local IDE development with remote cluster power

SUMMARY

An SDK that routes local PySpark code written in VS Code or PyCharm to execute on a remote Databricks cluster. Developers get full IntelliSense autocomplete, debugger support, and unit testing framework integration locally while actual Spark processing runs on the properly sized remote cluster.

Why we use it

Writing PySpark in a web browser notebook has no autocomplete, no step-through debugger, and no proper unit testing. Databricks Connect lets engineers use professional development tools they already know while keeping processing on a cluster with the right compute resources — local developer experience, cloud compute power.

new 2026

Domain 6 — Troubleshooting & Optimization

Spark UI, data skew, disk spill, AQE, broadcast join, tuning

Spark UI — Stages tab

Primary tool for diagnosing performance issues

SUMMARY

The Stages tab shows per-stage task metrics: task duration distribution (reveals skew), shuffle read/write bytes (reveals shuffle bottlenecks), spill to disk bytes (reveals memory pressure), GC time (reveals heap pressure), and input/output record counts.

Why we use it

Without the Stages tab, diagnosing why a job is slow requires guessing. The tab makes the bottleneck immediately visible: if 199 tasks complete in 2 seconds and 1 task takes 200 seconds, that is data skew — not a cluster sizing problem. Correct diagnosis leads to the correct fix rather than expensive trial-and-error.

new 2026

must know

Data skew

Why some tasks take far longer than others

SUMMARY

Data skew happens when some partitions have far more data than others after a shuffle. A few tasks process millions of rows while the other 195 tasks finish in seconds. The entire stage waits for the slowest task. Visible in Stages tab as extreme task duration imbalance.

Why we use it

Skew wastes cluster resources: 95% of executors are idle waiting for the 5% with oversized partitions. Enabling AQE (`spark.sql.adaptive.enabled=true`) detects and splits skewed partitions automatically at runtime. For extreme cases, salting adds a random number suffix to the skewed join key to artificially spread data across more partitions.

COMPARE

AQE (AUTOMATIC)

`spark.sql.adaptive.enabled=true` splits skewed partitions at runtime. Works for most cases.

SALTING (MANUAL)

Append random number suffix to skewed join key. For extreme skew that AQE cannot handle.

new 2026

must know

Disk spill

When executor memory runs out during processing

SUMMARY

Spill occurs when executor heap is insufficient to hold all data for a sort, join, or aggregation operation. Spark serialises the overflow to local executor disk and reads it back when needed — typically 10 to 100 times slower than in-memory processing. Visible as Spill (disk) greater than 0 in Stages tab.

Why we use it

A single spilling stage can turn a 30-minute job into a 3-hour job. The fix is either increasing `spark.executor.memory` so the data fits in RAM, or increasing `spark.sql.shuffle.partitions` so each partition is smaller and each individual task requires less memory to complete.

new 2026

must know

Adaptive Query Execution (AQE)

Runtime query plan optimisation

SUMMARY

AQE (`spark.sql.adaptive.enabled=true`, enabled by default in Spark 3+) collects actual runtime statistics during query execution and dynamically adjusts the execution plan: coalesces small shuffle partitions to reduce task overhead, detects and splits skewed join partitions, and switches from sort-merge to broadcast join when a table proves smaller than expected.

Why we use it

Static query planning is done before execution when data statistics may be stale or entirely unknown. AQE adapts mid-execution based on what the data actually looks like at runtime — automatically fixing problems that would otherwise require careful manual tuning and rerunning the query to test each change.

new 2026

shuffle.partitions tuning

Matching partition count to actual data size

SUMMARY

spark.sql.shuffle.partitions (default 200) controls how many partitions are created after a shuffle operation such as groupBy or join. Too many partitions on small data creates excessive task scheduling overhead that exceeds the actual computation time. Too few on large data causes spill or severe skew.

Why we use it

The default of 200 is designed for large multi-GB datasets. On a 50MB dataset, 200 partitions means each task processes 250KB — the overhead of JVM task launch, serialisation, and scheduling exceeds the actual work. Setting it to 4-8 for small datasets dramatically reduces job runtime. AQE automatically coalesces small shuffle partitions when enabled.

new 2026

Broadcast join

Eliminating shuffle for small table joins

SUMMARY

A broadcast join copies the smaller DataFrame to all executor nodes so each executor can join locally without shuffling the large table across the network. Triggered automatically when a table is under spark.sql.autoBroadcastJoinThreshold (default 10MB). Manually applied with broadcast(small_df) hint.

Why we use it

A sort-merge join on a 1TB orders table joined to a 5MB product lookup table shuffles all 1TB across the network — extremely expensive. A broadcast join sends only the 5MB lookup to every executor, eliminating the large-table shuffle entirely. The performance improvement can be 10-100x for this pattern.

must know

Domain 7 — Governance & Security

UC privilege model, column masking, row filters, Delta Sharing, Lakehouse Federation

Unity Catalog privilege model

Three levels — all three required to query a table

SUMMARY

To query a table in Unity Catalog, a user needs three privileges: USE CATALOG on the catalog, USE SCHEMA on the schema, and SELECT on the table. Missing any one of these blocks all access even if the other two are granted. DENY explicitly blocks access and overrides any GRANT from any source including group membership.

Why we use it

The layered model mirrors how data is organised — you navigate into a catalog, then a schema, then access a table. Each level is an access gate. This design allows admins to grant broad catalog access while restricting specific sensitive tables, or to DENY access to one group even within a broadly permissioned schema.

COMPARE

REVOKE	DENY
Removes a previously granted privilege. User may still have access via group membership.	Explicitly blocks access. Overrides any GRANT from any source — including group membership.

must know

exam trick

Managed vs external tables

What happens when you DROP TABLE

SUMMARY

Managed tables have their data lifecycle fully controlled by Unity Catalog — dropping the table permanently deletes both the catalog metadata and the underlying Parquet data files from cloud storage. External tables (created with LOCATION keyword) have data at an external path — dropping only removes the catalog metadata; data files at the location are completely unaffected.

Why we use it

This distinction is critical for production safety. Accidentally dropping a managed table means all data is gone permanently with no recovery. External tables are safer for shared datasets accessed by multiple systems — Databricks manages the catalog entry and query interface but does not own or control the underlying data files.

COMPARE

MANAGED	EXTERNAL (LOCATION=...)
DROP TABLE = catalog metadata + all data files deleted permanently. UC owns everything.	DROP TABLE = catalog metadata only removed. Data files remain at external location.

must know

exam trick

Column masking

Hiding sensitive column values at query time

SUMMARY

A SQL function applied to a column at the exact moment a query executes. The function receives the actual column value and returns either the real value (for privileged users matching a condition) or a masked substitute such as NULL, a partial value, or a redacted string for all other users.

Why we use it

Without column masking, the only way to hide a column from some users is to remove it entirely from the table they access — requiring duplicate tables or complex view maintenance for every sensitivity level and user group. Column masking applies the rule dynamically at query time so one physical table serves all users with appropriate column visibility per user identity.

new 2026

must know

Row-level security (row filters)

Different users see different rows from the same table

SUMMARY

A boolean SQL function applied at query time that filters which rows each user can see based on their identity, group membership, or custom attributes. Finance team members querying a shared table see only Finance department rows. All other users see no Finance rows from the same query.

Why we use it

Without row filters, the only option is creating separate physical tables per user group, or building complex UNION-based views. Row filters are completely transparent to end users — they query the same table with the same SQL and automatically receive only the rows they are authorised to see without any awareness of the filtering.

new 2026

must know

ABAC policies

Centralised fine-grained access control at scale

SUMMARY

Attribute-Based Access Control combines row filters and column masks into reusable, centrally managed policies based on user attributes like group membership, custom tags, or data classification labels. A single policy can automatically apply the same access rules across many tables simultaneously.

Why we use it

Without ABAC, every new table containing sensitive data requires manually re-applying the same row filter and column mask functions — error-prone and impossible to maintain at scale. ABAC centralises the policy so tagging a table as 'sensitive_hr' automatically applies all relevant access rules. One policy change propagates instantly to all tagged tables.

new 2026

Delta Sharing

Live data sharing without copying data

SUMMARY

An open protocol that shares live Delta tables with external recipients without copying the underlying data. Recipients use open-source client libraries for Python, pandas, Spark, or Power BI — no Databricks account required. Recipients can only read — they cannot write back, update, or delete any data.

Why we use it

Traditional data sharing copies data: storage cost doubles, the copy goes stale the moment it is created, and the recipient is perpetually behind the source. Delta Sharing eliminates the copy entirely — the recipient queries the live source table directly. One version of truth, always current, zero duplication cost, and the provider retains full control.

COMPARE

TRADITIONAL SHARING

Copy data to recipient. Stale immediately. Double storage. Sync lag always present.

DELTA SHARING

Zero copy. Recipient queries live data. Always current. No duplication cost.

new 2026

must know

Lakehouse Federation

Query external databases without moving data into Databricks

SUMMARY

Creates foreign catalogs in Unity Catalog that map to external database systems such as MySQL, PostgreSQL, or Snowflake. Users query external tables with standard Databricks SQL as if they were native Delta tables. Predicate pushdown sends WHERE clauses to the source database to minimise data transferred to Databricks.

Why we use it

ETL-based integration copies data from the source into Databricks before querying — adding storage cost, ingestion pipeline complexity, and data freshness lag. Lakehouse Federation queries the external source directly at runtime without any copy. Ideal for reference tables, lookup data, or any dataset where the external system remains the authoritative source.

new 2026